07-structuring-code-examples/11/shared_examples/spec/pass_block_spec.rb
```
require 'tempfile'

RSpec.describe FileKVStore do
  it_behaves_like 'KV store' do
    let(:tempfile) { Tempfile.new('kv.store') }
    let(:kv_store) { FileKVStore.new(tempfile.path) }
  end
end
```

With this technique, your block can contain whatever RSpec constructs you need. Here, we've used a let definition, but you can also also add helper methods and hooks inside the same kind of block.

To make this technique work, you'd need to change the definition of the KV store shared example group.

07-structuring-code-examples/11/shared_examples/spec/pass_block_spec.rb
```
RSpec.shared_examples 'KV store' do
  it 'allows you to fetch previously stored values' do
    kv_store.store(:language, 'Ruby')
    kv_store.store(:os, 'linux')

    expect(kv_store.fetch(:language)).to eq 'Ruby'
    expect(kv_store.fetch(:os)).to eq 'linux'
  end

  # remainder of examples...
end
```

The shared group no longer needs to take a block argument or define let(:kv_store). It just uses kv_store normally inside each example, trusting that the host group will define it with the right value for the context.

Your Turn

In this chapter, we took a second look at the main ways to structure your examples into groups. You saw how important it is to take the time to get the words right, and use terms in your specs like context or specify where they make more sense than describe or it.

We also dove into ways to move duplicated setup code out of your specs: let definitions, hooks, and helper methods. Although we've had brief encounters with these techniques in prior chapters, you saw how to get the most out of them here.

Finally, we talked about how to share examples or entire contexts. Now, it's time to bring these techniques to bear on your own code.